

## 6. 网络编程套接字

### 网络编程基础

#### 为什么需要网络编程？——丰富的网络资源

用户在浏览器中，打开在线视频网站，如优酷看视频，实质是通过网络，获取到网络上的一个视频资源。



与本地打开视频文件类似，只是视频文件这个资源的来源是网络。

相比本地资源来说，网络提供了更为丰富的网络资源：

#### 网络资源

视频资源

图片资源

文本资源

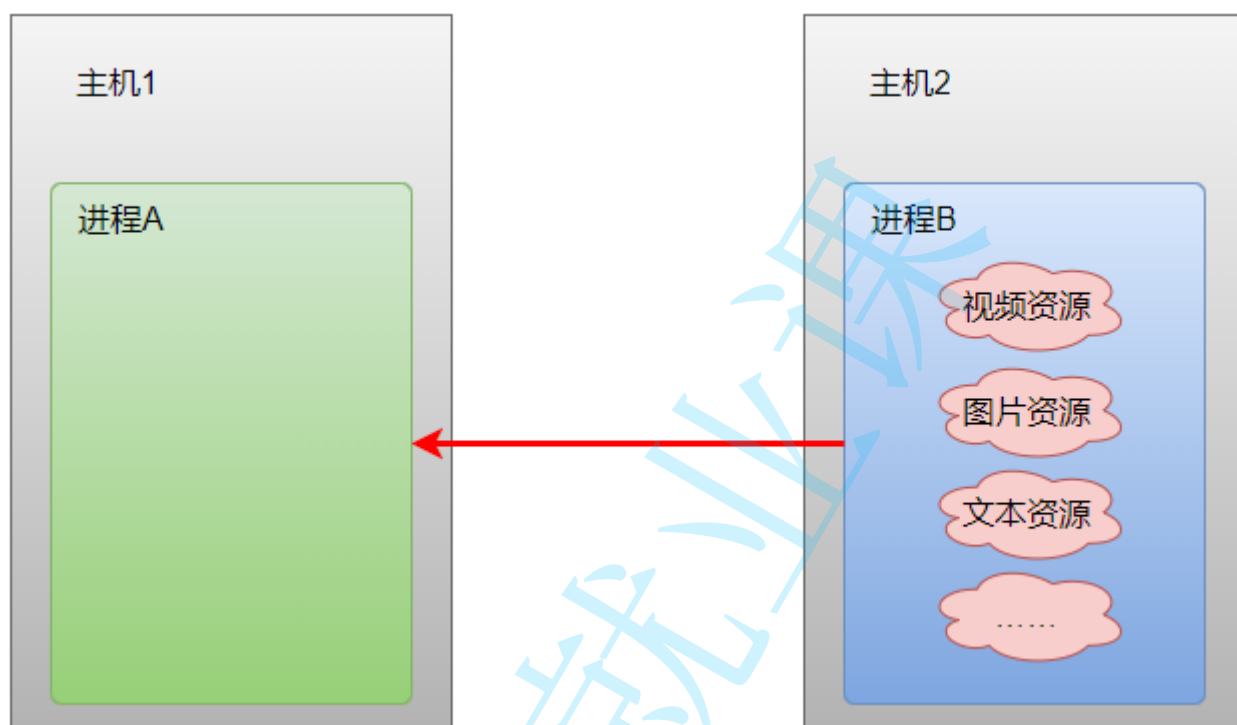
文本资源

所谓的**网络资源**，其实就是在网络中可以获取的各种数据资源。

而所有的网络资源，都是通过网络编程来进行数据传输的。

## 什么是网络编程

网络编程，指网络上的主机，通过**不同的进程**，以编程的方式实现**网络通信**（或称为**网络数据传输**）。



当然，我们只要满足**进程**不同就行；所以即便是同一个主机，只要是不同进程，基于网络来传输数据，也属于网络编程。

特殊的，对于开发来说，在条件有限的情况下，一般也都是在同一个主机中运行多个进程来完成网络编程。

但是，我们一定要明确，我们的目的是提供网络上不同主机，基于网络来传输数据资源：

- 进程A：编程来获取网络资源
- 进程B：编程来提供网络资源

## 网络编程中的基本概念

### 发送端和接收端

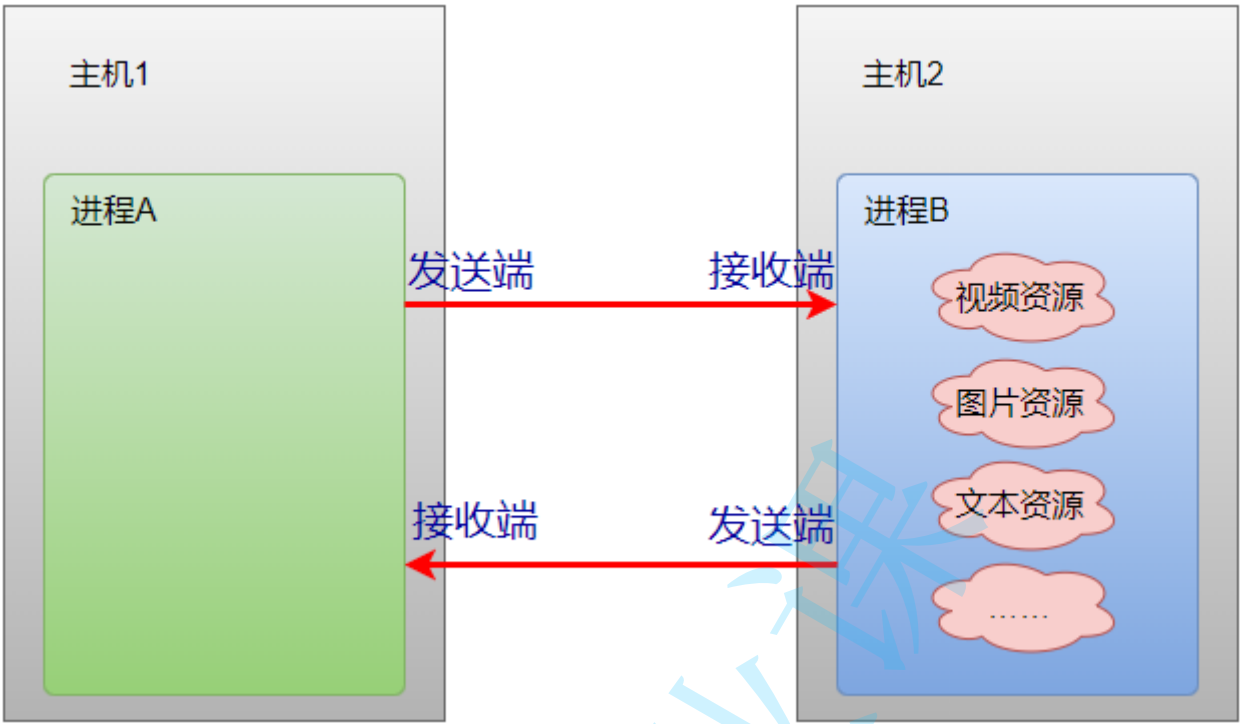
在一次网络数据传输时：

**发送端**：数据的**发送方进程**，称为发送端。发送端主机即网络通信中的源主机。

**接收端**：数据的**接收方进程**，称为接收端。接收端主机即网络通信中的目的主机。

**收发端**：发送端和接收端两端，也简称为收发端。

注意：发送端和接收端只是相对的，只是一次网络数据传输产生数据流向后的概念。



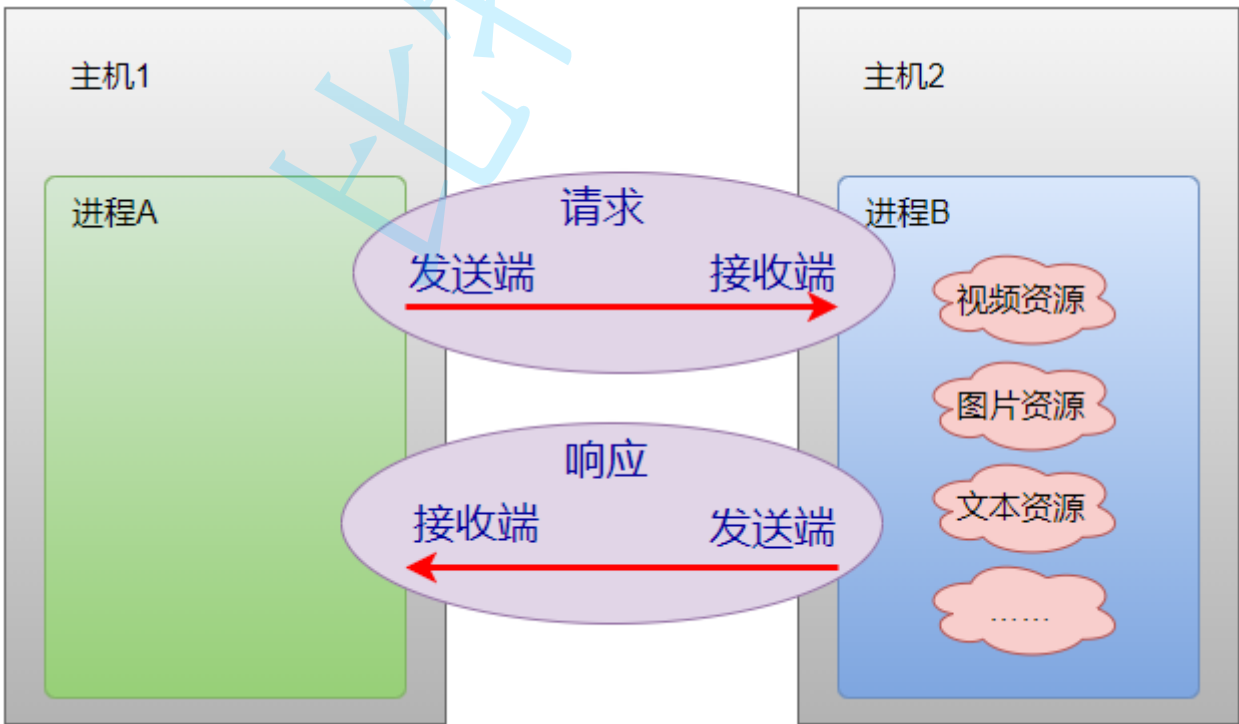
请求和响应

一般来说，获取一个网络资源，涉及到两次网络数据传输：

- 第一次：请求数据的发送
- 第二次：响应数据的发送。

好比在快餐店点一份炒饭：

先要发起请求：点一份炒饭，再有快餐店提供的对应响应：提供一份炒饭



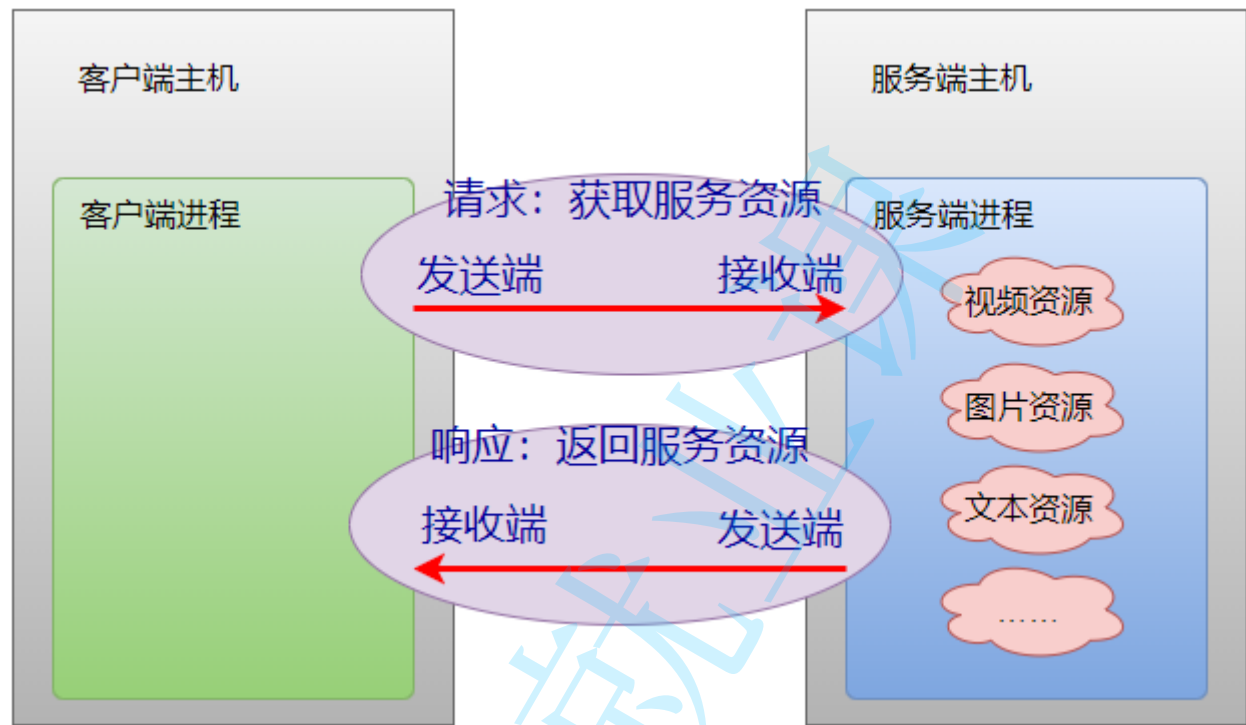
客户端和服务端

**服务端：**在常见的网络数据传输场景下，把**提供服务**的一方进程，称为服务端，可以提供对外服务。

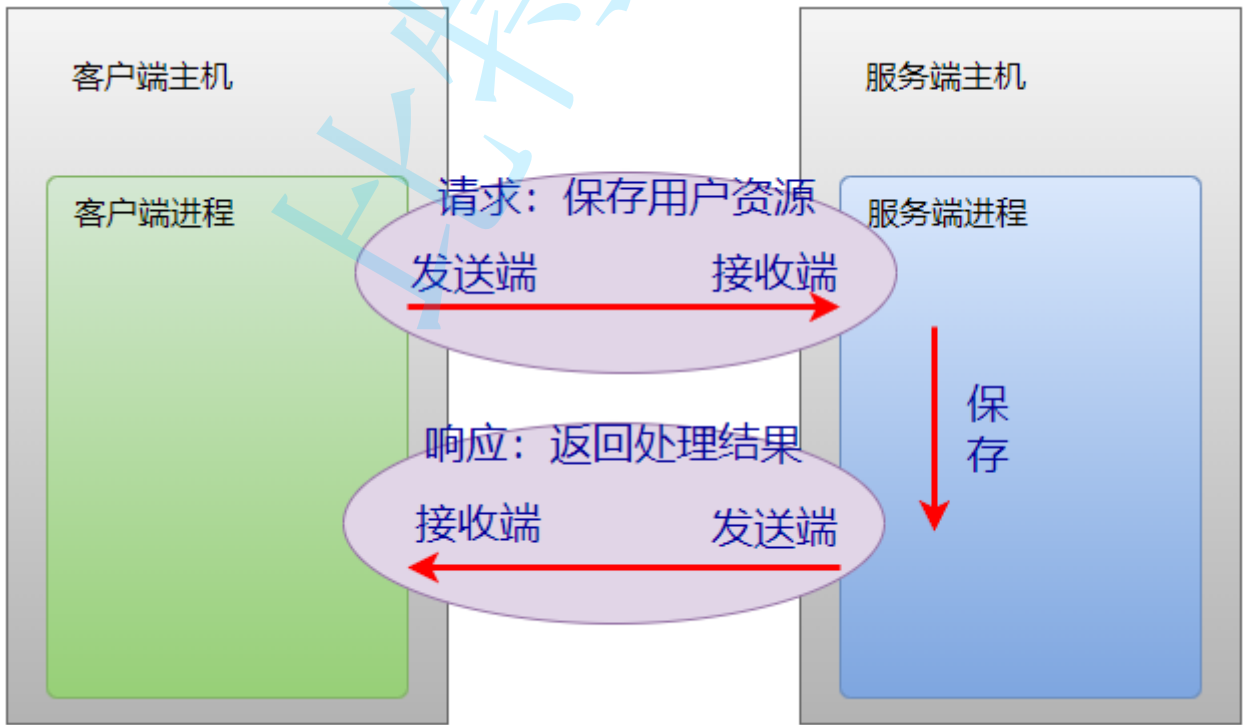
**客户端：****获取服务**的一方进程，称为客户端。

对于服务来说，一般是提供：

- 客户端获取服务资源



- 客户端保存资源在服务端



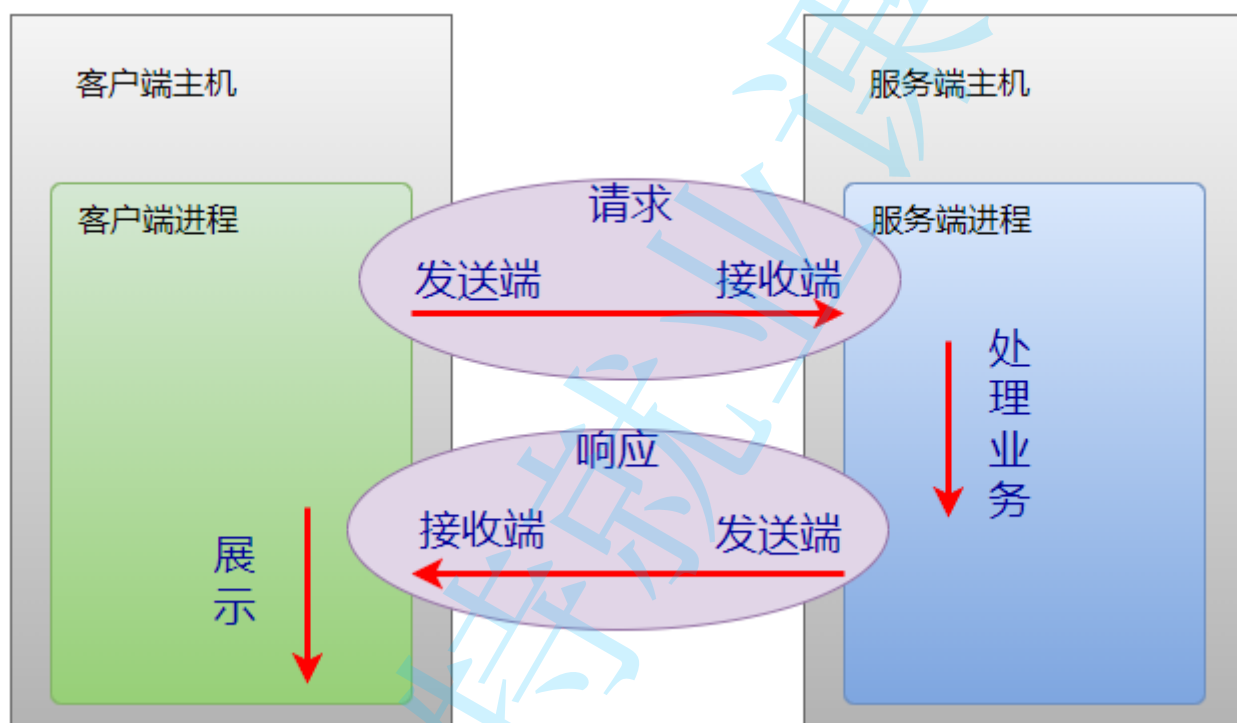
好比在银行办事：

- 银行提供存款服务：用户（客户端）保存资源（现金）在银行（服务端）
- 银行提供取款服务：用户（客户端）获取服务端资源（银行替用户保管的现金）

## 常见的客户端服务端模型

最常见的场景，客户端是指给用户使用的程序，服务端是提供用户服务的程序：

1. 客户端先发送请求到服务端
2. 服务端根据请求数据，执行相应的业务处理
3. 服务端返回响应：发送业务处理结果
4. 客户端根据响应数据，展示处理结果（展示获取的资源，或提示保存资源的处理结果）



## Socket套接字

### 概念

Socket套接字，是由系统提供用于网络通信的技术，是基于TCP/IP协议的网络通信的基本操作单元。基于Socket套接字的网络程序开发就是网络编程。

### 分类

Socket套接字主要针对传输层协议划分为如下三类：

**流套接字：**使用传输层TCP协议

TCP，即Transmission Control Protocol（传输控制协议），传输层协议。

以下为TCP的特点（细节后续再学习）：

- 有连接

- 可靠传输
- 面向字节流
- 有接收缓冲区，也有发送缓冲区
- 大小不限

对于字节流来说，可以简单的理解为，传输数据是基于IO流，流式数据的特征就是在IO流没有关闭的情况下，是无边界的数据，可以多次发送，也可以分开多次接收。

**数据报套接字：**使用传输层UDP协议

UDP，即User Datagram Protocol（用户数据报协议），传输层协议。

以下为UDP的特点（细节后续再学习）：

- 无连接
- 不可靠传输
- 面向数据报
- 有接收缓冲区，无发送缓冲区
- 大小受限：一次最多传输64k

对于数据报来说，可以简单的理解为，传输数据是一块一块的，发送一块数据假如100个字节，必须一次发送，接收也必须一次接收100个字节，而不能分100次，每次接收1个字节。

**原始套接字**

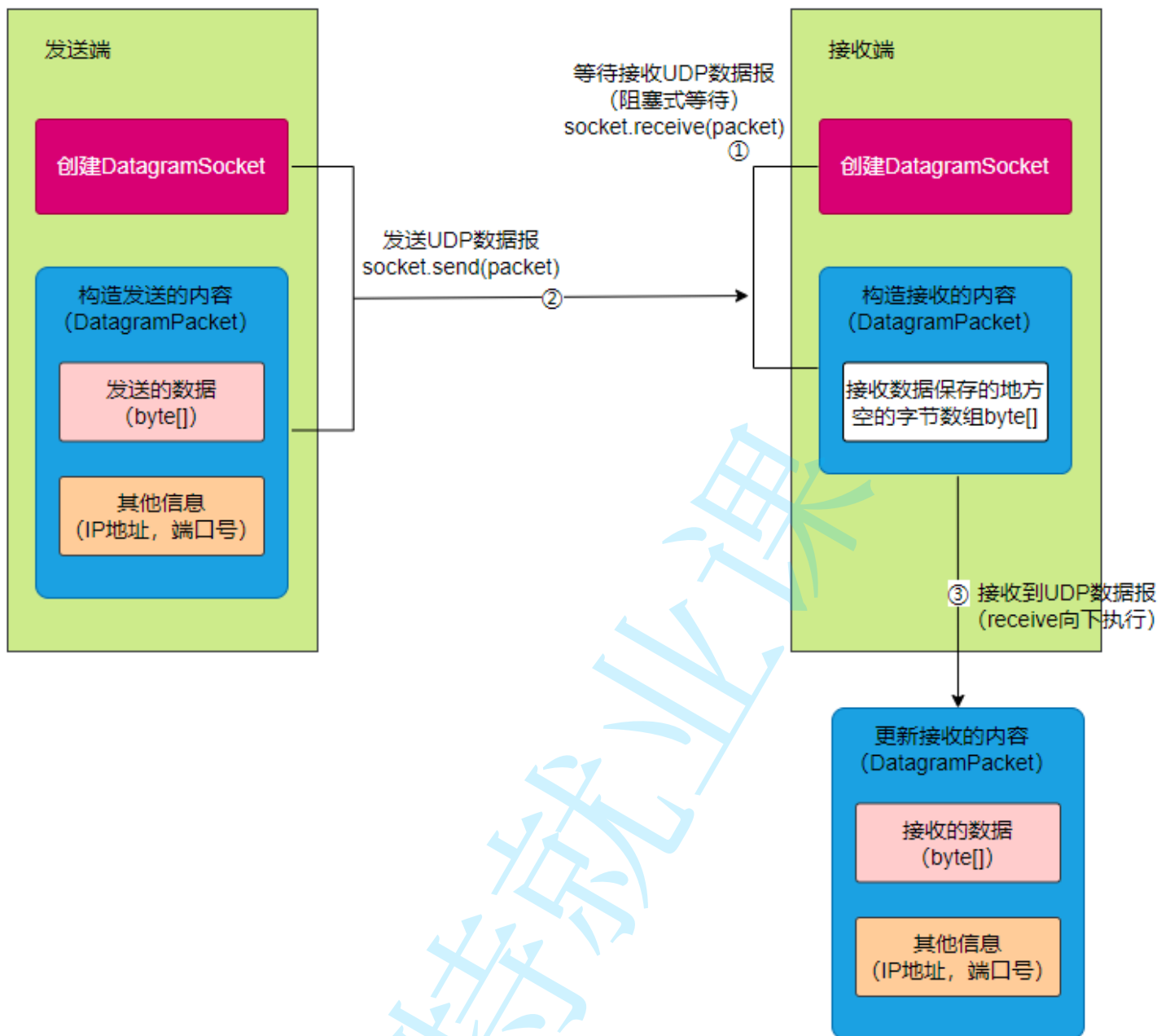
原始套接字用于自定义传输层协议，用于读写内核没有处理的IP协议数据。

我们不学习原始套接字，简单了解即可。

## Java数据报套接字通信模型

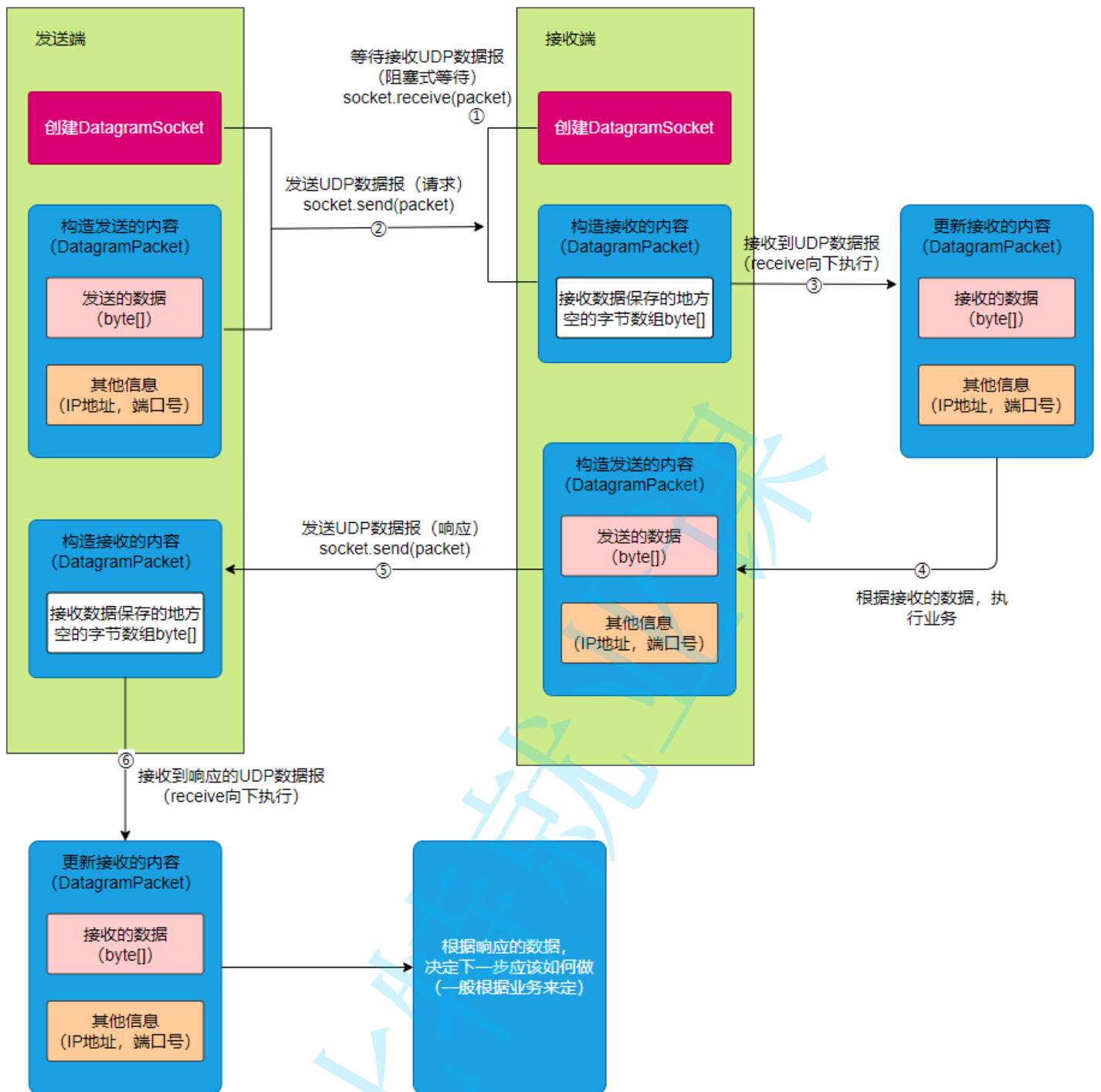
对于UDP协议来说，具有无连接，面向数据报的特征，即每次都是没有建立连接，并且一次发送全部数据报，一次接收全部的数据报。

java中使用UDP协议通信，主要基于 `DatagramSocket` 类来创建数据报套接字，并使用 `DatagramPacket` 作为发送或接收的UDP数据报。对于一次发送及接收UDP数据报的流程如下：



以上只是一次发送端的UDP数据报发送，及接收端的数据报接收，并没有返回的数据。也就是只有请求，没有响应。对于一个服务端来说，重要的是提供多个客户端的请求处理及响应，流程如下：

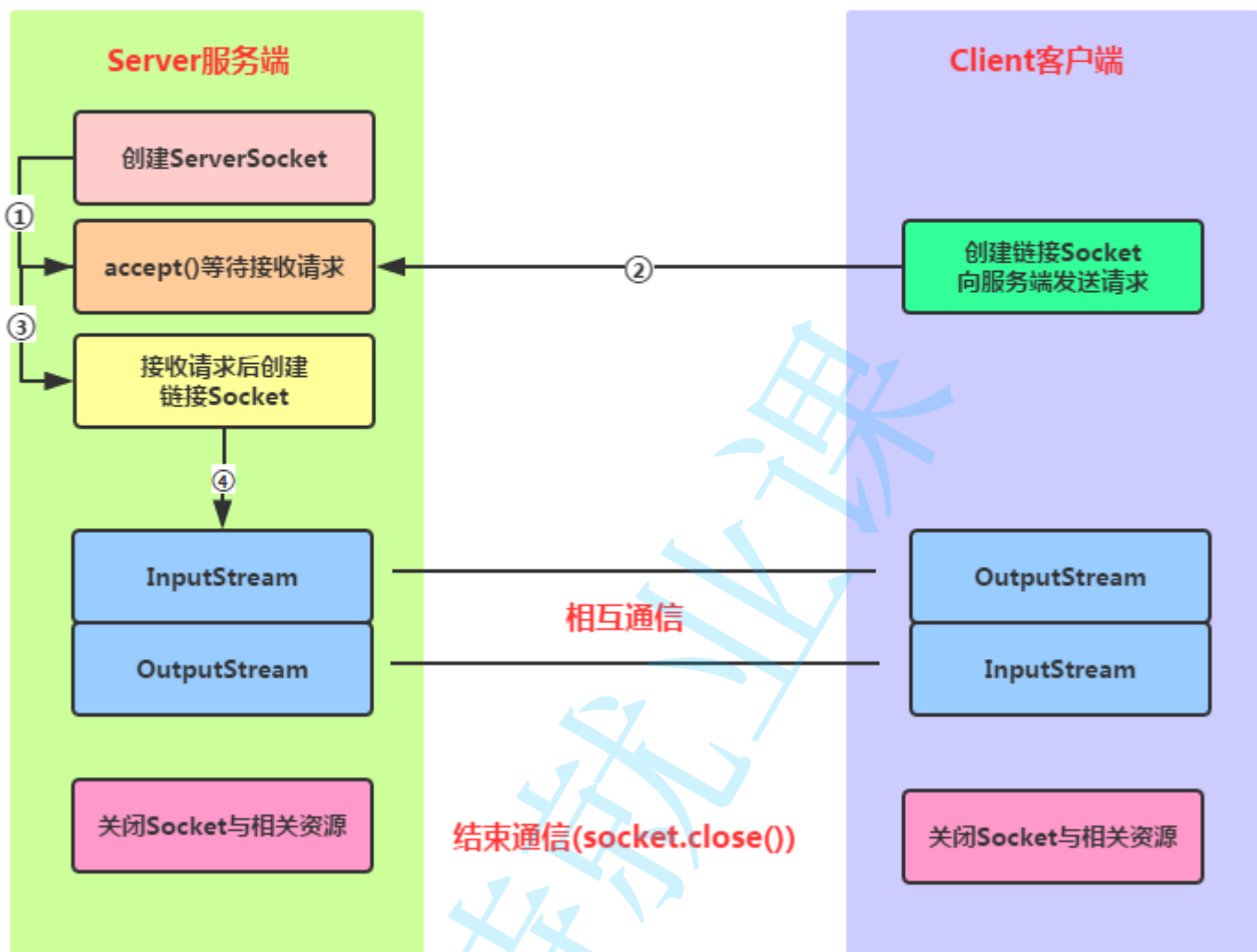




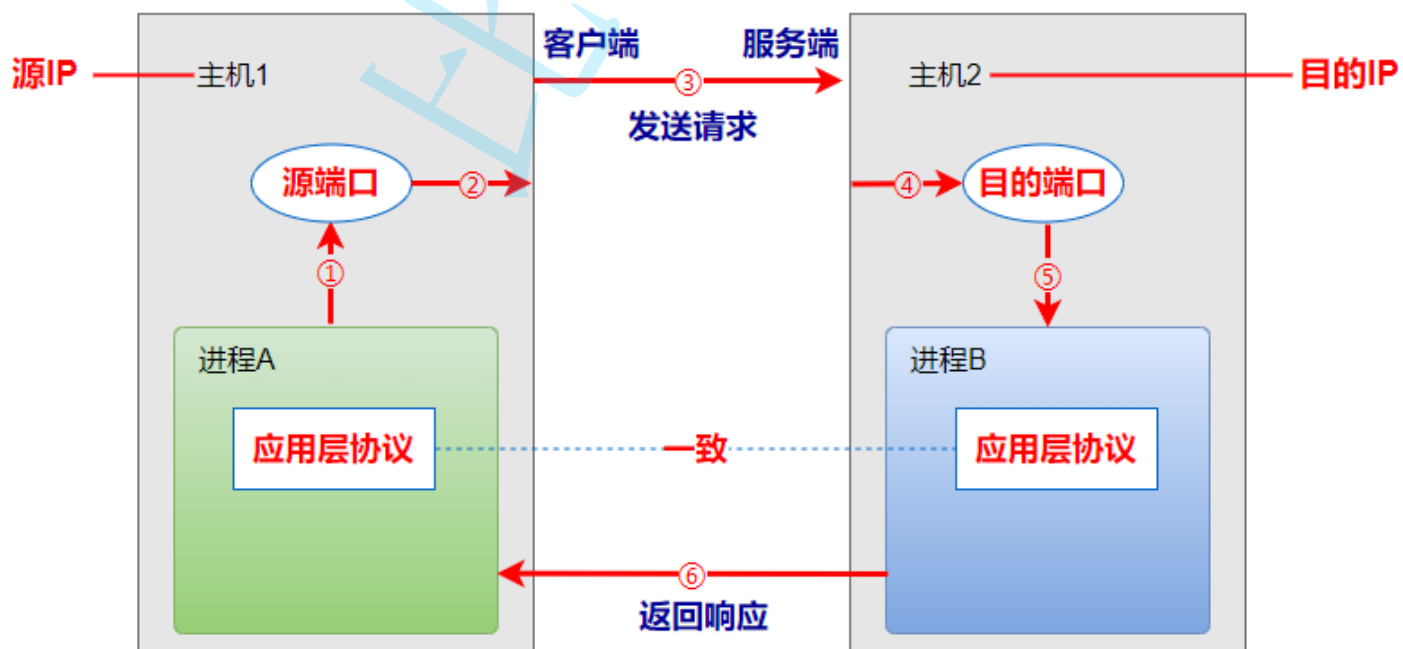
Java流套接字通信模型



## Socket通信模型



## Socket编程注意事项



1. 客户端和服务端：开发时，经常是基于一个主机开启两个进程作为客户端和服务端，但真实的场景，一般都是不同主机。
2. 注意目的IP和目的端口号，标识了一次数据传输时要发送数据的终点主机和进程
3. Socket编程我们是使用流套接字和数据报套接字，基于传输层的TCP或UDP协议，但应用层协议，也需要考虑，这块我们在后续来说明如何设计应用层协议。
4. 关于端口被占用的问题
5. 如果一个进程A已经绑定了一个端口，再启动一个进程B绑定该端口，就会报错，这种情况也叫端口被占用。对于java进程来说，端口被占用的常见报错信息如下：

```
Connected to the target VM, address: '127.0.0.1:50252', transport: 'socket'
Exception in thread "main" java.net.BindException: Address already in use JVM_Bind
    at java.net.DualStackPlainSocketImpl.bind0(Native Method)
    at java.net.DualStackPlainSocketImpl.socketBind(DualStackPlainSocketImpl.java:102)
    at java.net.AbstractPlainSocketImpl.bind(AbstractPlainSocketImpl.java:513)
    at java.net.PlainSocketImpl.bind(PlainSocketImpl.java:180)
    at java.net.ServerSocket.bind(ServerSocket.java:375)
    at java.net.ServerSocket.<init>(ServerSocket.java:237)
    at java.net.ServerSocket.<init>(ServerSocket.java:128)
    at org.example.tcp.demo3.TcpServer.main(TcpServer.java:19)
```

地址已经被占用，就是说的端口

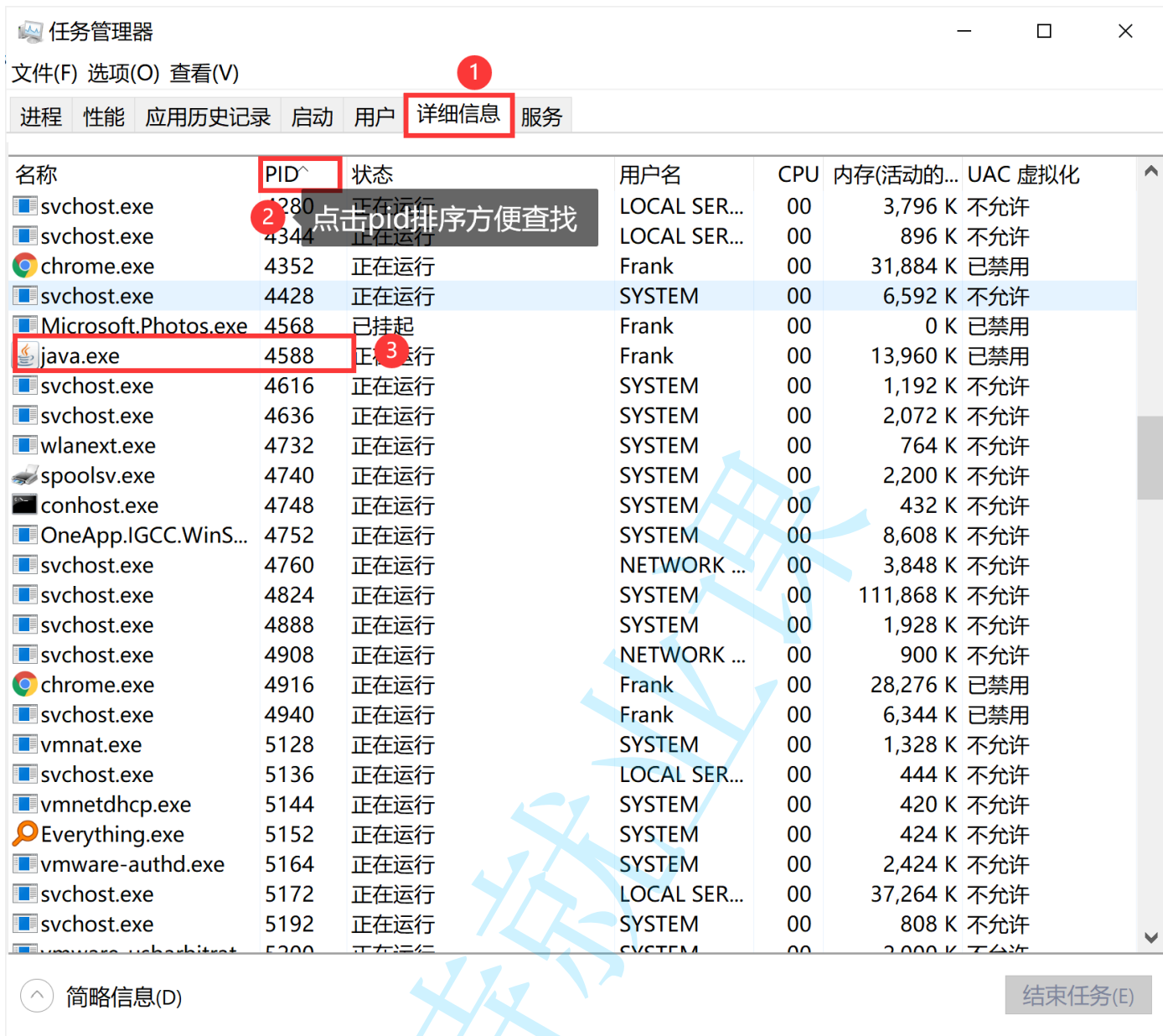
此时需要检查进程B绑定的是哪个端口，再查看该端口被哪个进程占用。以下为通过端口号查进程的方式：

- 在cmd输入 `netstat -ano | findstr 端口号`，则可以显示对应进程的pid。如以下命令显示了8888进程的pid

```
C:\Users\Frank>netstat -ano | findstr 8888
TCP    0.0.0.0:8888        0.0.0.0:0          LISTENING    4588
TCP    [::]:8888          [::]:0             LISTENING    4588
```

进程的pid

- 在任务管理器中，通过pid查找进程



解决端口被占用的问题：

- 如果占用端口的进程A不需要运行，就可以关闭A后，再启动需要绑定该端口的进程B
- 如果需要运行A进程，则可以修改进程B的绑定端口，换为其他没有使用的端口。

## UDP数据报套接字编程

### API 介绍

#### DatagramSocket

`DatagramSocket` 是UDP Socket，用于发送和接收UDP数据报。

`DatagramSocket` 构造方法：

| 方法签名                          | 方法说明 |
|-------------------------------|------|
| <code>DatagramSocket()</code> |      |

|                          |                                             |
|--------------------------|---------------------------------------------|
|                          | 创建一个UDP数据报套接字的Socket，绑定到本机任意一个随机端口（一般用于客户端） |
| DatagramSocket(int port) | 创建一个UDP数据报套接字的Socket，绑定到本机指定的端口（一般用于服务端）    |

DatagramSocket 方法：

| 方法签名                           | 方法说明                            |
|--------------------------------|---------------------------------|
| void receive(DatagramPacket p) | 从此套接字接收数据报（如果没有接收到数据报，该方法会阻塞等待） |
| void send(DatagramPacket p)    | 从此套接字发送数据报包（不会阻塞等待，直接发送）        |
| void close()                   | 关闭此数据报套接字                       |

DatagramPacket

DatagramPacket是UDP Socket发送和接收的数据报。

DatagramPacket 构造方法：

| 方法签名                                                                      | 方法说明                                                                                       |
|---------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| DatagramPacket(byte[] buf, int length)                                    | 构造一个DatagramPacket以用来接收数据报，接收的数据保存在字节数组（第一个参数buf）中，接收指定长度（第二个参数length）                     |
| DatagramPacket(byte[] buf, int offset, int length, SocketAddress address) | 构造一个DatagramPacket以用来发送数据报，发送的数据为字节数组（第一个参数buf）中，从0到指定长度（第二个参数length）。address指定目的主机的IP和端口号 |

DatagramPacket 方法：

| 方法签名                     | 方法说明                                       |
|--------------------------|--------------------------------------------|
| InetAddress getAddress() | 从接收的数据报中，获取发送端主机IP地址；或从发送的数据报中，获取接收端主机IP地址 |
| int getPort()            | 从接收的数据报中，获取发送端主机的端口号；或从发送的数据报中，获取接收端主机端口号  |

|                  |           |
|------------------|-----------|
| byte[] getData() | 获取数据报中的数据 |
|------------------|-----------|

构造UDP发送的数据报时，需要传入 `SocketAddress`，该对象可以使用 `InetSocketAddress` 来创建。

## InetSocketAddress

`InetSocketAddress`（`SocketAddress` 的子类）构造方法：

| 方法签名                                                       | 方法说明                    |
|------------------------------------------------------------|-------------------------|
| <code>InetSocketAddress(InetAddress addr, int port)</code> | 创建一个Socket地址，包含IP地址和端口号 |

## 代码示例

### UDP Echo Server

```
1 public class UdpEchoServer {
2     private DatagramSocket socket = null;
3
4     // 参数是服务器要绑定的端口
5     public UdpEchoServer(int port) throws SocketException {
6         socket = new DatagramSocket(port);
7     }
8
9     // 使用这个方法启动服务器。
10    public void start() throws IOException {
11        System.out.println("服务器启动!");
12        while (true) {
13            // 反复的，长期的执行针对客户端请求处理的逻辑。
14            // 一个服务器，运行过程中，要做的事情，主要是三个核心环节。
15            // 1. 读取请求，并解析
16            DatagramPacket requestPacket = new DatagramPacket(new byte[4096], 40
17            socket.receive(requestPacket);
18            // 这样的转字符串的前提是，后续客户端发的数据就是一个文本的字符串。
19            String request = new String(requestPacket.getData(), 0, requestPacke
20            // 2. 根据请求，计算出响应
21            String response = process(request);
22            // 3. 把响应写回给客户端
23            // 此时需要告知网卡，要发的内容是啥，要发给谁。
24            DatagramPacket responsePacket = new DatagramPacket(response.getBytes
25                requestPacket.getSocketAddress());
26            socket.send(responsePacket);
27            // 记录日志，方便观察程序执行效果。
```

```

28         System.out.printf("[%s:%d] req: %s, resp: %s\n", requestPacket.getAd
29             request, response);
30     }
31 }
32
33 // 根据请求计算响应。由于是回显程序，响应内容和请求完全一样。
34 public String process(String request) {
35     return request;
36 }
37
38 public static void main(String[] args) throws IOException {
39     UdpEchoServer server = new UdpEchoServer(9090);
40     server.start();
41 }
42 }

```

## UDP Echo Client

```

1 public class UdpEchoClient {
2     private DatagramSocket socket = null;
3     private String serverIp;
4     private int serverPort;
5
6     // 服务器的 ip 和 服务器的端口。
7     public UdpEchoClient(String ip, int port) throws SocketException {
8         serverIp = ip;
9         serverPort = port;
10        // 这个 new 操作，就不再指定端口了。让系统自动分配一个空闲端口。
11        socket = new DatagramSocket();
12    }
13
14    // 让这个客户端反复的从控制台读取用户输入的内容。把这个内容构造成 UDP 请求，发给服务器。
15    // 最终再显示在客户端的屏幕上。
16    public void start() throws IOException {
17        Scanner scanner = new Scanner(System.in);
18        System.out.println("客户端启动!");
19        while (true) {
20            // 1. 从控制台读取用户输入的内容
21            System.out.print("> "); // 命令提示符，提示用户要输入字符串。
22            String request = scanner.next();
23            // 2. 构造请求对象，并发给服务器。
24            DatagramPacket requestPacket = new DatagramPacket(request.getBytes()
25                InetAddress.getByIp(serverIp), serverPort);

```

```

26         socket.send(requestPacket);
27         // 3. 读取服务器的响应, 并解析出响应内容.
28         DatagramPacket responsePacket = new DatagramPacket(new byte[4096], 4
29         socket.receive(responsePacket);
30         String response = new String(responsePacket.getData(), 0, responsePa
31         // 4. 显示到屏幕上.
32         System.out.println(response);
33     }
34 }
35
36 public static void main(String[] args) throws IOException {
37     UdpEchoClient client = new UdpEchoClient("127.0.0.1", 9090);
38     // UdpEchoClient client = new UdpEchoClient("42.192.83.143", 9090);
39     client.start();
40 }
41 }

```

## UDP Dict Server

编写一个英译汉的服务器. 只需要重写 process

```

1 public class UdpDictServer extends UdpEchoServer {
2     private Map<String, String> dict = new HashMap<>();
3
4     public UdpDictServer(int port) throws SocketException {
5         super(port);
6
7         dict.put("cat", "小猫");
8         dict.put("dog", "小狗");
9         dict.put("fuck", "卧槽");
10        // 可以在这里继续添加千千万万个单词. 使每个单词都有一个对应的翻译.
11    }
12
13    // 是要复用之前的代码, 但是又要做出调整.
14    @Override
15    public String process(String request) {
16        // 把请求对应单词的翻译, 给返回回去.
17        return dict.getOrDefault(request, "该词没有查询到!");
18    }
19
20    public static void main(String[] args) throws IOException {
21        UdpDictServer server = new UdpDictServer(9090);
22        // start 不需要重新再写一遍了. 直接就复用了之前的 start !
23        server.start();

```



```
24     }
25 }
```

## TCP流套接字编程

和刚才UDP类似. 实现一个简单的英译汉的功能

### API 介绍

#### ServerSocket

`ServerSocket` 是创建TCP服务端Socket的API。

`ServerSocket` 构造方法：

| 方法签名                                | 方法说明                       |
|-------------------------------------|----------------------------|
| <code>ServerSocket(int port)</code> | 创建一个服务端流套接字Socket，并绑定到指定端口 |

`ServerSocket` 方法：

| 方法签名                         | 方法说明                                                                  |
|------------------------------|-----------------------------------------------------------------------|
| <code>Socket accept()</code> | 开始监听指定端口（创建时绑定的端口），有客户端连接后，返回一个服务端Socket对象，并基于该Socket建立与客户端的连接，否则阻塞等待 |
| <code>void close()</code>    | 关闭此套接字                                                                |

#### Socket

`Socket` 是客户端Socket，或服务端中接收到客户端建立连接（`accept`方法）的请求后，返回的服务端Socket。

不管是客户端还是服务端Socket，都是双方建立连接以后，保存的对端信息，及用来与对方收发数据的。

`Socket` 构造方法：

| 方法签名                                       | 方法说明                                     |
|--------------------------------------------|------------------------------------------|
| <code>Socket(String host, int port)</code> | 创建一个客户端流套接字Socket，并与对应IP的主机上，对应端口的进程建立连接 |

## Socket 方法:

| 方法签名                           | 方法说明        |
|--------------------------------|-------------|
| InetAddress getAddress()       | 返回套接字所连接的地址 |
| InputStream getInputStream()   | 返回此套接字的输入流  |
| OutputStream getOutputStream() | 返回此套接字的输出流  |

## 代码示例

## TCP Echo Server

```

1 public class TcpEchoServer {
2     private ServerSocket serverSocket = null;
3
4     // 这个操作就会绑定端口号
5     public TcpEchoServer(int port) throws IOException {
6         serverSocket = new ServerSocket(port);
7     }
8
9     // 启动服务器
10    public void start() throws IOException {
11        System.out.println("服务器启动!");
12        while (true) {
13            Socket clientSocket = serverSocket.accept();
14            processConnection(clientSocket);
15        }
16    }
17
18    // 通过这个方法来处理一个连接的逻辑。
19    private void processConnection(Socket clientSocket) {
20        System.out.printf("[%s:%d] 客户端上线!\n", clientSocket.getInetAddress().
21        // 接下来就可以读取请求，根据请求计算响应，返回响应三步走了。
22        // Socket 对象内部包含了两个字节流对象，可以把这两字节流对象获取到，完成后续的读
23        try (InputStream inputStream = clientSocket.getInputStream();
24            OutputStream outputStream = clientSocket.getOutputStream()) {
25            // 一次连接中，可能会涉及到多次请求/响应
26            while (true) {
27                // 1. 读取请求并解析。为了读取方便，直接使用 Scanner。

```

```

28         Scanner scanner = new Scanner(inputStream);
29         if (!scanner.hasNext()) {
30             // 读取完毕，客户端下线。
31             System.out.printf("[%s:%d] 客户端下线!\n", clientSocket.getIn
32                 break;
33         }
34         // 这个代码暗含一个约定，客户端发过来的请求，得是文本数据，同时，还得带
35         String request = scanner.next();
36         // 2. 根据请求计算响应
37         String response = process(request);
38         // 3. 把响应写回给客户端。把 OutputStream 使用 PrintWriter 包裹一
39         PrintWriter writer = new PrintWriter(outputStream);
40         // 使用 PrintWriter 的 println 方法，把响应返回给客户端。
41         // 此处用 println，而不是 print 就是为了在结尾加上 \n。方便客户端
42         writer.println(response);
43         // 这里还需要加一个 "刷新缓冲区" 操作。
44         writer.flush();
45
46         // 日志，打印当前的请求详情。
47         System.out.printf("[%s:%d] req: %s, resp: %s\n", clientSocket.ge
48             request, response);
49     }
50 } catch (IOException e) {
51     e.printStackTrace();
52 } finally {
53     // 在 finally 中加上 close 操作，确保当前 socket 被及时关闭!!
54     try {
55         clientSocket.close();
56     } catch (IOException e) {
57         e.printStackTrace();
58     }
59 }
60 }
61
62 public String process(String request) {
63     return request;
64 }
65
66 public static void main(String[] args) throws IOException {
67     TcpEchoServer server = new TcpEchoServer(9090);
68     server.start();
69 }
70 }

```

# TCP Echo Client

```
1 public class TcpEchoClient {
2     private Socket socket = null;
3
4     // 要和服务器通信，就需要先知道，服务器所在的位置。
5     public TcpEchoClient(String serverIp, int serverPort) throws IOException {
6         // 这个 new 操作完成之后，就完成了 tcp 连接的建立。
7         socket = new Socket(serverIp, serverPort);
8     }
9
10    public void start() {
11        System.out.println("客户端启动");
12
13        Scanner scannerConsole = new Scanner(System.in);
14
15        try (InputStream inputStream = socket.getInputStream();
16             OutputStream outputStream = socket.getOutputStream()) {
17            while (true) {
18                // 1. 从控制台输入字符串。
19                System.out.print("-> ");
20                String request = scannerConsole.next();
21                // 2. 把请求发送给服务器
22                PrintWriter printWriter = new PrintWriter(outputStream);
23                // 使用 println 带上换行。后续服务器读取请求，就可以使用 scanner.r
24                printWriter.println(request);
25                // 不要忘记 flush，确保数据是真的发送出去了!!
26                printWriter.flush();
27                // 3. 从服务器读取响应。
28                Scanner scannerNetwork = new Scanner(inputStream);
29                String response = scannerNetwork.next();
30                // 4. 把响应打印出来
31                System.out.println(response);
32            }
33        } catch (IOException e) {
34            e.printStackTrace();
35        }
36    }
37
38    public static void main(String[] args) throws IOException {
39        TcpEchoClient client = new TcpEchoClient("127.0.0.1", 9090);
40        client.start();
41    }
42 }
```

## 服务器引入多线程

如果只是单个线程, 无法同时响应多个客户端.

此处给每个客户端都分配一个线程.

```
1 // 启动服务器
2 public void start() throws IOException {
3     System.out.println("服务器启动!");
4     while (true) {
5         Socket clientSocket = serverSocket.accept();
6         Thread t = new Thread(() -> {
7             processConnection(clientSocket);
8         });
9         t.start();
10    }
11 }
```

## 服务器引入线程池

为了避免频繁创建销毁线程, 也可以引入线程池.

```
1 // 启动服务器
2 public void start() throws IOException {
3     System.out.println("服务器启动!");
4     ExecutorService service = Executors.newCachedThreadPool();
5     while (true) {
6         Socket clientSocket = serverSocket.accept();
7         // 使用线程池, 来解决上述问题
8         service.submit(new Runnable() {
9             @Override
10            public void run() {
11                processConnection(clientSocket);
12            }
13        });
14    }
15 }
```

## 长短连接

TCP发送数据时，需要先建立连接，什么时候关闭连接就决定是短连接还是长连接：

短连接：每次接收到数据并返回响应后，都关闭连接，即是短连接。也就是说，短连接只能一次收发数据。

长连接：不关闭连接，一直保持连接状态，双方不停的收发数据，即是长连接。也就是说，长连接可以多次收发数据。

对比以上长短连接，两者区别如下：

- 建立连接、关闭连接的耗时：短连接每次请求、响应都需要建立连接，关闭连接；而长连接只需要第一次建立连接，之后的请求、响应都可以直接传输。相对来说建立连接，关闭连接也是要耗时的，长连接效率更高。
- 主动发送请求不同：短连接一般是客户端主动向服务端发送请求；而长连接可以是客户端主动发送请求，也可以是服务端主动发。
- 两者的使用场景有不同：短连接适用于客户端请求频率不高的场景，如浏览网页等。长连接适用于客户端与服务端通信频繁的场景，如聊天室，实时游戏等。

扩展了解：

基于BIO（同步阻塞IO）的长连接会一直占用系统资源。对于并发要求很高的服务端系统来说，这样的消耗是不能承受的。

由于每个连接都需要不停的阻塞等待接收数据，所以每个连接都会在一个线程中运行。

一次阻塞等待对应着一次请求、响应，不停处理也就是长连接的特性：一直不关闭连接，不停的处理请求。

实际应用时，服务端一般是基于NIO（即同步非阻塞IO）来实现长连接，性能可以极大的提升。